LAB 16 - PROMPTS

Automation Stage: a Scheduled, Risk-Checked Forward Test and Live Paper Trading

Open your AI coding assistant (for example Claude Code or GitHub Copilot) in the labs_fullcode folder. Work down the steps: copy each prompt into the assistant, review the code it writes, then run the command. The complete, working code is already in Lab16-paper-trading-bot/solution/ if you want to compare. Never paste API keys into an AI chat.

Step 1: Build the risk layer and the kill switch (A7)

Prompt - copy into your AI coding assistant
Create labs_fullcode/pipeline/risk.py with class RiskManager(cfg). kill_switch_on() = the file named by risk.kill_switch_file exists under labs. check(broker, status, symbol, wants_buy, allow_queue=False) returns (ok_to_trade, ok_to_buy, notes): kill switch -> (False, False, ['KILL_SWITCH present - trading halted']); session 'closed' -> blocked with 'market closed' unless allow_queue (then note 'order will queue (GTC) for the next session'); equity / last_equity - 1 <= -risk.max_daily_loss_pct -> no new buys; symbol not in risk.allowed_symbols -> no buy; account trading_blocked -> no buy; otherwise ['all checks passed']. cap_weight(weight, equity, price) applies max_position_pct and max_order_value. kill_main(argv) for `python -m pipeline kill --on | --off | --status`: --on writes the file with a timestamp; --off MOVES it to _state/kill_switch_history/KILL_SWITCH.<timestamp> (an audit record - never delete it). Risk is deterministic code; no model may override it. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: python -m pipeline kill --status prints 'KILL SWITCH OFF'.

Step 2: Build one decision cycle (A7)

Prompt - copy into your AI coding assistant
Create labs_fullcode/pipeline/runner.py. decide(bars, cfg) = the configured strategy's target from pipeline.signals.target_position on COMPLETED bars only, returning bar_date, close, signal and (for ema_cross) ema_fast and ema_slow. run_cycle(broker, om, risk, bars, run_time, cfg, weight, allow_queue=False, poll_seconds=0): read position and pending shares from the pipeline.execution.OrderManager; wants_buy = signal 1 and nothing held or pending; wants_sell = signal 0 and shares held; run RiskManager.check; if allowed call om.rebalance({symbol: weight}) or om.rebalance({symbol: 0.0}) with tag = the bar date (one decision per bar, so a double-fired job cannot double-trade); otherwise action BLOCKED if a trade was wanted; return a log row with run_time, bar_date, symbol, close, ema_fast, ema_slow, signal, position_before, action, qty, order_id, status, fill_price, equity, checks. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | Open in ChatGPT

Check: run_cycle returns a dict with the 15 log fields.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 3: Build the forward-test replay and the logs (A8)

Prompt - copy into your AI coding assistant

In labs_fullcode/pipeline/runner.py add replay(cfg, start, end): offline forward test on PaperBrokerSim (state_file bot_sim.json, reset) - for each 2025 session set the simulator clock to 09:35, mark the price at today's open, run_cycle on bars up to YESTERDAY's close with weight risk.max_position_pct, then process the day's bar and roll end_of_day; stop if the kill switch appears. Write _outputs/trade_log.csv (fresh, one row per session) and _outputs/Lab16_equity_log.csv (date, equity, position, close), print every non-HOLD row, the number of orders, start and end equity and SPY buy and hold over the same days, and save chart Lab16_forward_test.png. once(cfg): one cycle; offline it acts at the open after the last bundled bar and appends to trade_log.csv; online it appends to _outputs/live_paper_log.csv. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Check: python -m pipeline run --help lists --once, --loop, --live-paper, --interval, --max-cycles, --start and --end.

Step 4: Add the live paper loop and scheduling (A7)

Prompt - copy into your AI coding assistant

In labs_fullcode/pipeline/runner.py add live_loop(cfg, interval, max_cycles): on the Alpaca PAPER account (or the simulator with a clear message if no keys) run up to max_cycles cycles: print the clock line, stop with 'KILL SWITCH ON - no orders sent; loop stopped' if the kill switch exists, run_cycle with weight live_paper.max_position_pct, allow_queue=True and poll_seconds live_paper.poll_seconds online (the OrderManager sends extended-hours limit orders outside regular hours), append to _outputs/live_paper_log.csv, print the row and every position with unrealised P&L, then sleep interval seconds; finish with the dashboard address. main(argv): --loop --live-paper -> live_loop; --loop offline -> replay; --loop online -> live_loop; otherwise once. schedule_main(argv) for `python -m pipeline schedule`: print and write to _outputs/Lab16_schedule/ a crontab line with CRON_TZ=America/New_York for schedule.run_at_exchange_time Mon-Fri (with the Singapore time in US summer and winter), a launchd plist (com.lioncitycapital.pipeline, WorkingDirectory labs) and a Windows schtasks command, each running `python -m pipeline run --once` from labs and appending to _outputs/scheduler.log. Do not install anything. Rules: run from the labs folder; reuse labs_fullcode/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Open in Claude | **Open in ChatGPT**

Check: python -m pipeline schedule --help works.

Step 5: Run the 2025 forward test (A8, K7)

Do this (no prompt needed)

Run: python -m pipeline run --loop (same as: python3 Lab16-paper-trading-bot/run_stage.py --loop). Offline this replays 2025 day by day; each order fills at that day's open in the simulator.

Check: 250 sessions, 5 orders: BUY 159 after the 2025-01-21 close, SELL after 2025-02-27, BUY 166 after 2025-05-02, SELL after 2025-11-19, BUY 163 after 2025-11-28; equity USD 100,000.00 -> 116,249.79 (+16.25%) versus SPY buy and hold +17.06%; trade_log.csv with 250 rows; Lab16_forward_test.png.

Step 6: Audit the decision log (A8)

Prompt - copy into your AI coding assistant

Read labs_fullcode/_outputs/trade_log.csv and write an audit summary for the compliance officer: number of sessions, decisions by action, every order with its bar date, fill price and equity after the fill, and any row where a risk check did not say 'all checks passed'. Do not change any file.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Open in Claude | Open in ChatGPT

Check: 250 rows: 5 BUY/SELL rows with status filled and 245 HOLD rows; 249 rows say 'all checks passed' and one (2025-01-27) says 'daily loss -2.05% breaches 2% limit - no new buys' - a HOLD day with the position already held, so the rule blocked nothing.

Step 7: Test the kill switch (A7)

Do this (no prompt needed)
Run: python -m pipeline kill --on, then python -m pipeline run --once, then python -m pipeline kill --off.

Check: The --once row shows checks 'KILL_SWITCH present - trading halted' (no order can be sent); --off prints 'moved to _state/kill_switch_history/KILL_SWITCH.<timestamp>' - the halt is kept as an audit record.

Step 8: Run the strategy LIVE on the paper account (A7, A8, K7)

Do this (no prompt needed)
With paper keys in .env, during class run: python -m pipeline run --loop --interval 60 --live-paper --max-cycles 5. Watch decision -> order -> fill -> position -> log for each cycle. After two or three cycles, in a SECOND terminal run python -m pipeline kill --on and watch the next cycle halt. Then run python -m pipeline kill --off and open _outputs/live_paper_log.csv and the Alpaca dashboard.

Check: Cycle 1 sends one SPY buy for 10% of equity (an extended-hours LIMIT DAY order in Singapore class hours; about 14 shares on USD 100,000) and prints its lifecycle and the position; later cycles HOLD ('buy order ... still working' or position held); after kill --on: 'KILL SWITCH ON - no orders sent; loop stopped'. Without keys the same loop runs on the simulator (at a weekend the buy queues as a GTC limit order).

Step 9: Schedule the runner (A7)

Do this (no prompt needed)
Run: python -m pipeline schedule and read the three entries. Only install one if your instructor approves (crontab -e, launchctl load, or schtasks in a Command Prompt).

Check: One cycle at 09:35 New York = 21:35 Singapore (US summer) / 22:35 (US winter), Mon-Fri; files crontab.txt, com.lioncitycapital.pipeline.plist and schtasks.cmd in _outputs/Lab16_schedule/; nothing is installed.

Run it

Step	Command / file
Run Lab 16	python3 Lab16-paper-trading-bot/run_stage.py (same as: python -m pipeline run)
The complete code	pipeline/risk.py, pipeline/runner.py (reference copy: Lab16-paper-trading-bot/solution/)
Outputs	_outputs/ (charts, CSV files and reports written by the stage)

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.